

Plan Técnico de Implementación

Plataforma contable unificada sobre **Modux** — reemplazo de Visual IVA, Visual Conta y Visual Sueldos + unificación del sistema de gestión existente

Estudio contable · Plazo objetivo: 4 meses · Backend: Modux (PHP 8.2+ / MySQL 8) · Documento de referencia para evaluación técnica

CONTENIDO

1. Objetivo y alcance técnico

2. Situación actual (4 sistemas)

3. Arquitectura objetivo

4. Stack tecnológico — Modux

5. Modelo de datos

6. Módulos a implementar

7. Integración entre módulos
8. Migración y unificación de datos

9. Integraciones externas y API

10. Seguridad y multi-tenancy

11. Infraestructura y despliegue

12. Evolución futura (El horizonte)

13. Plan de ejecución (4 meses)

14. Riesgos y mitigaciones

15. Supuestos y dependencias

1. Objetivo y alcance técnico

Reemplazar las tres aplicaciones de escritorio (Delphi/VCL + Firebird) por una **plataforma web única, modular y multi-tenant** construida sobre **Modux**, unificando IVA, contabilidad y sueldos con el sistema de gestión del estudio ya existente, sobre una base de datos central, y habilitando la automatización y la futura comercialización como SaaS.

DENTRO DE ALCANCE (4 MESES)	FUERA DE ALCANCE (EVOLUCIÓN POSTERIOR)
• Backend Modux + base de datos central multi-tenant • Migración de datos de los 3 sistemas Firebird • Acoplamiento del sistema de gestión existente (vía API) • Módulos IVA, Contable y Sueldos operativos • Reportes y formularios oficiales esenciales • Automatizaciones iniciales + portal · puesta en marcha	• Paridad legal fina de Sueldos (convenios particulares, casos borde) • Migración completa del sistema de gestión a módulo Modux • Comunicaciones omnicanal, IA (RAG), conciliación bancaria • Empaquetado/venta SaaS (billing, planes) — habilitado por la arquitectura • Módulos de flota/RRHH del sistema actual (no son del dominio contable)

SERVICIOS DE TERCEROS — NO INCLUIDOS EN EL HONORARIO DE DESARROLLO

El desarrollo cubre la plataforma y los módulos. Quedan **por fuera** (los provee/abona el cliente o se cotizan aparte, con costos propios y recurrentes): **infraestructura cloud/hosting y backups**, **WhatsApp Business API** (alta y costo por conversación de Meta), **IA** (desarrollo y consumo de tokens), **funnels/envío de email** (herramienta y campañas) y el **dominio** (ya disponible). Las capacidades de "El horizonte" (§12) son evolución posterior y se cotizan por separado.

2. Situación actual (4 sistemas)

El estudio opera hoy con **cuatro** sistemas que no comparten datos: tres de escritorio (Logosoft, motor Firebird, licencia de proveedor externo) y uno web propio.

SISTEMA	TIPO	MOTOR / VERSIÓN	TABLAS	ALCANCE FUNCIONAL
Visual IVA 6.10.25	Escritorio	Firebird 1.5 / ODS 10.1	33	IVA compras/ventas, retenciones, libros, DDJJ/SIAP
Visual Conta 5.00.14	Escritorio	Firebird 1.5 / ODS 10.1	31	Asientos, diario/mayor, balances, activos fijos
Visual Sueldos 5.80.26.1	Escritorio	Firebird 2.x / ODS 11.2	103	Liquidaciones, recibos, F931, LSD, F1357, convenios
Sistema de Gestión ("Synergys")	Web	Laravel 12 / MySQL	—	CRM de clientes, vencimientos, tareas, facilidades ARCA, BCRA, DFE

EL 4° SISTEMA — ESTRATEGIA

El sistema de gestión web ya existe (Laravel 12 + MySQL) y **funciona**. No se reescribe en los 4 meses: se **acopla vía API** a la plataforma Modux (los datos de clientes/contribuyentes se unifican y comparten), y su **migración** a un módulo Modux nativo queda como evolución posterior. Sus módulos de **flota/ambulancias y RRHH** quedan fuera de alcance (no son del dominio contable).

3. Arquitectura objetivo

Modux es un **monolito modular API-first** (respuestas siempre JSON), con inyección de dependencias explícita (PSR-11, sin facades ni estado global) y **multi-tenancy nativa** a nivel de fila. El frontend es un cliente web que consume la API.

CONSUMIDORES

SPA web nueva (Vue 3 + Vite) · Portal del cliente · API para terceros · sistema de gestión actual (durante el acoplamiento)

BACKEND — MODUX

Kernel → **Pipeline global** (CORS · SecurityHeaders · RequestSize · RateLimit · Logger) → **Middlewares de ruta** (Auth JWT · Tenant · Permission · Scope · Entitlement) → **Controller** (inyección por tipo)

MÓDULOS DE DOMINIO

IVA Contable Sueldos Gestión/CRM ApiKeys Billing* IA* (* opcionales)

SERVICIOS CORE (FRAMEWORK)

Container PSR-11 · EventDispatcher · Job Queue (DB) · Migraciones versionadas · FormRequest/Validator · Reportes (PDF/Excel)

DATOS

MySQL 8 / MariaDB (InnoDB, utf8mb4) · **aislamiento por tenant_id** a nivel de fila · cache/colas sobre DB



INTEGRACIONES

AFIP/ARCA · BCRA · Email/SMTP · WhatsApp (API oficial) · webhooks firmados (HMAC) · (futuro) bancos / facturadores

INFRAESTRUCTURA

Docker · CI/CD con quality gate (PHPStan L6 · PHPCS PSR-12 · tests) · worker queue:work (systemd/supervisord) · /health · backups diarios

Principios de diseño (Modux)

- **Cero magia:** sin facades ni service locator; toda dependencia se inyecta por constructor y es testeable.
- **Modularidad:** cada dominio es un módulo autodescubierto (`app/Modules/*`) con Controllers/Services/Repositories/Requests/routes propios.
- **Dato único:** cliente/empresa, CUIT y plan de cuentas viven una vez y los consumen todos los módulos.
- **Multi-tenant de fábrica:** `TenantMiddleware` + claim `tenant_id` en el JWT + filtrado por fila → base directa para la reventa SaaS.
- **Ciclo de request explícito y auditable:** pipeline de middlewares inmutable, excepciones tipadas → HTTP, logger estructurado JSON.

4. Stack tecnológico — Modux

CAPA / COMPONENTE	TECNOLOGÍA	APORTE
Backend / API	Modux (PHP 8.2+)	Monolito modular propio, API JSON, sin dependencia de framework externo
Base de datos	MySQL 8 / MariaDB (PDO crudo)	Control total de cada query; destino de la migración
Inyección de dependencias	Container PSR-11 (autowiring + <code>makeWith</code>)	Dependencias explícitas, código testeable
Autenticación	JWT + refresh tokens con rotación · API keys (scopes)	Sesiones seguras + acceso server-to-server
Autorización	RBAC (<code>PermissionMiddleware</code>) + scopes + entitlements	Permisos por operación, gating por feature de tenant
Validación	<code>FormRequest</code> + Validator (valida en construcción → 422)	Previene mass-assignment; reglas declarativas
Asincronía	Job Queue (DB) + <code>EventDispatcher</code> síncrono	Reportes, importaciones y notificaciones en background
Migraciones	Versionadas con batch (<code>php modux migrate</code> / <code>rollback</code> / <code>fresh</code>)	Esquema reproducible y seguro en cada deploy
Reportes	PDF + planillas Excel	Libros, recibos, comprobantes, listados
Calidad	PHPStan nivel 6 · PHPCS PSR-12 · suite de tests + CI gate	Base estable, no se publica nada roto
Frontend	SPA nueva — Vue 3 + Vite + Tailwind + Pinia	Consume la API Modux por JWT; totalmente desacoplada del backend

POR QUÉ MODUX Y NO UN FRAMEWORK EXTERNO

Es un framework **propio** diseñado para APIs controladas, herramientas internas y **SaaS multi-tenant** — exactamente este caso. Sin licencias de terceros, ciclo de request totalmente transparente, multi-tenancy / entitlements / billing de fábrica, y un *quality gate* (static analysis + tests) que protege la base que luego se revende.

5. Modelo de datos

Esquema central en MySQL, por dominios. Núcleo común compartido + tablas por módulo. Toda tabla operativa lleva `tenant_id` `CHAR(36)` para el aislamiento multi-tenant. Migraciones versionadas con `php modux migrate`.

Núcleo común (compartido)

ENTIDAD	ROL
<code>tenants</code>	Unidad de aislamiento (estudio / cliente del SaaS)
<code>empresas</code>	Empresa/contribuyente administrado dentro de un tenant
<code>clientes</code> / <code>personas</code>	Contribuyentes y terceros (clientes, proveedores) — dato único compartido con el CRM
<code>plan_cuentas</code>	Plan de cuentas compartido por Contable e IVA
<code>periodos</code>	Períodos fiscales/contables con estado (abierto/cerrado)
<code>usuarios</code> / <code>roles_permisos</code>	Acceso y RBAC
<code>auditoria</code>	Trazas de operaciones sensibles

Inventario por módulo (tablas representativas)

MÓDULO	TABLAS CLAVE (ORIGEN)	ORIGEN
IVA	<code>COMPRAS</code> , <code>COMPRAS_DISCRIMINACION</code> , <code>VENTAS</code> , <code>VENTA_DISCRIMINACION</code> , <code>RETENCIONES</code> , <code>RETENC_COMPRAS</code> , <code>CONDICION_IVA</code> , <code>TIPO_COMPROBANTE</code> , <code>RUBROS</code> , <code>PROVEEDORES</code> , <code>CLIENTES</code>	Firebird · 33 tablas
Contable	<code>CUENTAS</code> , <code>ASIENTOS</code> , <code>DETALLE_ASIENTOS</code> , <code>ASIENTOS_AUTOMATICOS</code> , <code>ASIENTOS_MODELO</code> , <code>PERIODOS</code> , <code>ACF_ACTIVO_TB</code> , <code>ACF_DEPRECAF_TM</code>	Firebird · 31 tablas
Sueldos	<code>PERSONAL</code> , <code>LIQUIDACIONES</code> , <code>CONCEPTOS</code> , <code>CONCEPTOS_LIQUIDACION</code> , <code>RECIBOS</code> , <code>F1357_*</code> , <code>LSD_*</code> , <code>CONTRIBUCIONES_*</code> , <code>TABLA_GANANCIAS</code> , <code>UOCRA*</code> , <code>FAECYS_CATEGORIAS</code> , <code>EMBARGOS_EMPLEADOS</code> , <code>VACACIONES</code>	Firebird · 103 tablas
Gestión/CRM	<code>personas</code> , <code>tributos</code> , <code>vencimientos</code> , <code>tasks*</code> , <code>facilidad_plan</code> , <code>bcra_*</code> , <code>notificaciones_dfe</code>	MySQL (Laravel) · acoplar

NOTA DE ESQUEMA

El esquema real de los sistemas Firebird se obtiene de **producción**, no de los instaladores (se autoactualizan vía UpdateDB.exe; puede haber *drift*). Se reconcilia con diff de esquema antes de cargar (§8). El esquema del sistema de gestión actual está parcialmente desincronizado con sus migraciones → su fuente de verdad también es la base de producción.

6. Módulos a implementar

Cada dominio se genera con `php modux make:module <Nombre> --with-tenant` (scaffolding de Controllers/Repositories/Services/Requests/routes, tenant-scoped) y se autodescubre al boot.

6.1 · Módulo IVA reemplaza Visual IVA

- Comprobantes de compra/venta con discriminación de alícuotas, neto, exento, impuestos internos.
- Retenciones y percepciones; libros de IVA, subdiarios, listados.
- DDJJ / exportaciones AFIP-SIAP (TXT) e importación de Mis Comprobantes.
- Asiento contable automático hacia Contable (vía `EventDispatcher`, §7).
- Referencia: **64 reportes** del sistema actual ya mapeados (dataset → campos → tablas).

6.2 · Módulo Contable reemplaza Visual Conta

- Plan de cuentas jerárquico; asientos manuales, **automáticos** y por **modelo/póliza**.
- Libro Diario, Libro Mayor, Balance de sumas y saldos, Estado de Resultados en tiempo real.
- Activos fijos y depreciación; cierre de período/ejercicio con bloqueo.

6.3 · Módulo Sueldos reemplaza Visual Sueldos

- Legajo de personal, conceptos configurables, liquidaciones y recibos.
- **F931, LSD, F1357**; ganancias 4ª categoría; convenios (Comercio/FAECYS, UOCRA); embargos, vacaciones.
- Asiento de sueldos automático hacia Contable.

RIESGO REGULATORIO

Sueldos es el módulo más extenso y sensible. La **paridad legal fina** (convenios particulares, casos borde de F931/LSD) se completa como ajuste posterior al período de 4 meses.

6.4 · Módulo Gestión / CRM acoplar (existe en Laravel)

Clientes, tributos, vencimientos con workflow por etapas, tareas, facilidades ARCA, BCRA, DFE. **No se reescribe** en los 4 meses: se **acopla vía API** (Modux expone/consume endpoints; se reutiliza la capa de import/sync existente) para unificar el maestro de clientes/CUIT. Migración a módulo Modux nativo = evolución posterior.

6.5 · Estimación de esfuerzo por módulo

Estimación de horas de desarrollo, criterio conservador, para **paridad funcional completa**. El compromiso de 4 meses entrega la plataforma operativa difiriendo la paridad fina de Sueldos. Modux acelera la base (auth, RBAC, multi-tenant, migraciones y colas vienen

de fábrica), por lo que el esfuerzo se concentra en la lógica de negocio.

COMPONENTE	HORAS (EST.)	ENTREGA
Configuración de plataforma Modux + núcleo común (Modux aporta auth/RBAC/tenant/migraciones/colas)	80 h	Mes 1
Migración de datos — 3 bases Firebird (pipeline ya construido)	120 h	Meses 1 y 4
Acoplamiento del sistema de gestión existente (API / eventos / sync)	60 h	Meses 1–2
Integración entre módulos + automatizaciones + portal	40 h	Meses 2–4
Módulo IVA	200 h	Mes 2
Módulo Contable	240 h	Mes 3
Módulo Sueldos (paridad completa)	500 h	Meses 3–4 + ajuste posterior
Frontend SPA (Vue 3) — cobertura completa de los 4 módulos	400 h	Meses 1–4 + posterior
Total — paridad funcional completa (backend + SPA)	~1.640 h	—

LECTURA DEL ESFUERZO

La paridad funcional completa (**backend + SPA**) se estima en **~1.640 h** (~10 meses full-time de un desarrollador). El compromiso de **4 meses** entrega el **núcleo operativo**: backend con los tres módulos funcionando, sistema de gestión acoplado y las **pantallas core del SPA**. La cobertura completa del SPA, la **paridad legal fina de Sueldos** y la **migración nativa del CRM** se completan como ajuste posterior (§14).

7. Integración entre módulos

El valor central frente a los sistemas aislados: el dato se carga una vez y los asientos se generan solos. Se implementa con el `EventDispatcher` de Modux (eventos de dominio) + `DB::withTransaction()`.

Módulo IVA · registra/confirma un comprobante → `dispatch('comprobante.confirmado')`



Listener en Contable · mapea comprobante → cuentas (reglas por tipo/condición) y crea el asiento en transacción



Módulo Sueldos · liquidación cerrada → `dispatch('liquidacion.cerrada')` → asiento de sueldos

ORIGEN	EVENTO	RESULTADO EN CONTABLE
IVA — Ventas	comprobante.confirmado	Asiento de ventas (deudores, IVA débito, ventas)
IVA — Compras	comprobante.confirmado	Asiento de compras (compras, IVA crédito, proveedores)
Sueldos	liquidacion.cerrada	Asiento de sueldos a pagar y cargas sociales

8. Migración y unificación de datos

Cuatro orígenes: **3 bases Firebird** (corte one-shot, pipeline ya construido y validado) + **1 base MySQL** del sistema de gestión (acoplar primero, migrar después).

8.1 · Bases Firebird → MySQL (toolchain propio)

HERRAMIENTA	FUNCIÓN	ESTADO
fbschema_diff.py	Diff de esquema Firebird (referencia vs producción)	listo
fb2mysql.py	DDL Firebird (isql -x) → esquema MySQL (tipos, dominios, generadores, computed)	listo
fbdata2mysql.py	Datos del .fdb → INSERTs MySQL (driver fdb)	listo
validar_migracion.py	Conteo de filas + reconciliación de importes (exit 1 si difiere)	listo
Motores FB portables	fb15 (ODS 10.1) y fb25 (ODS 11.2)	listo

Procedimiento (dos pasadas): (1) reconocimiento remoto; (2) **ensayo** con backup `gbak -b -g` → restore → diff → conversión → carga → validación; (3) **corte real** en freeze de fin de período; (4) viejo en solo-lectura como respaldo.

RIESGO	MITIGACIÓN (RESUELTA)
Drift de esquema (prod ≠ demo)	fbschema_diff.py antes de cargar
Encoding (IVA declara WIN1251 erróneo)	Conexión NONE + decode cp1252
Generadores con valor 0	sql_mode=NO_AUTO_VALUE_ON_ZERO
Copia en caliente del .fdb	Siempre gbak , nunca copia directa
"Importó sin error" ≠ correcto	Reconciliación financiera por período

8.2 · Sistema de gestión (MySQL) → unificación

Al ser MySQL → MySQL es de bajo riesgo. Enfoque: definir un **maestro único de clientes/empresas** y unificar; durante el acoplamiento, sincronización vía API (Modux API keys + los *ImportController / SyncController existentes). Migración del esquema a

9. Integraciones externas y API

SERVICIO	USO	MECANISMO MODUX
AFIP / ARCA	Vencimientos, Mis Comprobantes, padrón, facilidades, SIAP	Conectores HTTP + Job Queue
BCRA	Consulta de deudores	Conector HTTP
Email / SMTP	Notificaciones, recibos, reportes	EmailHelper + colas
WhatsApp	Avisos, solicitudes de documentación	API oficial recomendada
Terceros / sistema de gestión	Acoplamiento e integraciones a medida	API keys (scopes) + webhooks firmados (HMAC, anti-replay)

API-FIRST

Modux expone todo como API JSON con autenticación por JWT o API key (scoped por tenant) y verificación de webhooks por firma HMAC. Esto habilita el acoplamiento del 4º sistema y las futuras integraciones (bancos, facturadores, portal) sin tocar el núcleo.

10. Seguridad y multi-tenancy

EJE	MEDIDA (NATIVA EN MODUX)
Autenticación	JWT + refresh tokens con rotación (single-use, revocables); rate limiting de login
Autorización	RBAC por permiso (<code>PermissionMiddleware</code>) + scopes de API key
Multi-tenancy	Aislamiento por <code>tenant_id</code> a nivel de fila (<code>TenantMiddleware</code> + claim JWT)
Feature gating	Entitlements + quotas por tenant (402/429 accionables) — base del SaaS
Integridad contable	Asientos no editables tras cierre de período; cierre de ejercicio con bloqueo
Transporte y headers	HTTPS/TLS · <code>SecurityHeadersMiddleware</code> · límite de tamaño de request
Auditoría	Logger estructurado JSON + trazas de operaciones sensibles
Respaldo	Backups diarios automáticos + recuperación ante fallos

11. Infraestructura y despliegue

- **Docker** (app PHP + MySQL) para entornos reproducibles (dev / staging / prod).
- **CI/CD con quality gate:** `composer validate` + PHPCS PSR-12 + PHPStan nivel 6 + tests en cada push (hook pre-push local + CI).
- **Migraciones** seguras en cada deploy (`php modux migrate` , idempotente, con rollback).
- **Worker de colas** (`php modux queue:work`) gestionado por **systemd/supervisord** (reinicio automático).
- **Endpoint `/health`** para probes de balanceador y monitoreo de disponibilidad.
- **Backups diarios** automáticos + pruebas de restore; logs centralizados.

12. Evolución futura (El horizonte)

Capacidades que la arquitectura Modux habilita. No incluidas en los 4 meses; varias ya tienen soporte de framework.

CAPACIDAD	HABILITADOR EN MODUX
Comunicaciones unificadas (WhatsApp, email, voz, chat)	Módulo de mensajería omnicanal sobre la base única, ligado al expediente
Asistente con IA / RAG sobre los datos del estudio	Add-on <code>cynchro/modux-ia</code> (LLM + RAG, sin Python/Node) — gating por entitlement <code>ia.rag</code>
Comercialización SaaS (planes, cobro, cuotas)	Add-on <code>cynchro/modux-billing</code> (Stripe / MercadoPago) + entitlements + usage metering
Portal de autogestión del cliente	Roles externos + scopes + acceso acotado por tenant
Automatizaciones configurables	EventDispatcher + Job Queue programada
Conciliación bancaria / cobranza de honorarios	Importadores + webhooks de pago firmados
API abierta para terceros	API keys con scopes (ya nativo)

VENTAJA PARA LA REVENTA SAAS

La multi-tenancy, los entitlements/cuotas y el billing (Stripe/MercadoPago) son **parte del framework**. El producto nace listo para venderse a otros estudios como servicio, sin reingeniería: solo definir planes y activar el módulo de billing.

13. Plan de ejecución (4 meses)

MES	HITO	ENTREGABLES	LIBERA
1	Plataforma Modux + migración + acoplamiento	Backend base (auth/RBAC/tenant), esquema MySQL, ensayo de migración validado, acoplamiento del sistema de gestión (maestro de clientes unificado)	Cuota 1
2	Módulo IVA + integración contable	Comprobantes, libros, exportaciones AFIP, asiento automático IVA → Contable	Cuota 2
3	Módulo Contable + inicio Sueldos	Asientos, diario/mayor, balances; legajo y conceptos de Sueldos	Cuota 3
4	Sueldos + corte real + puesta en marcha	Liquidaciones/recibos/F931/LSD, asiento de sueldos, cutover de datos, validación final, go-live	Cuota 4

METODOLOGÍA

Iterativa, con entregables verificables al cierre de cada mes (cada cuota se libera contra su hito). El corte real de datos se ejecuta sobre fin de período para minimizar riesgo.

14. Riesgos y mitigaciones

RIESGO	IMPACTO	MITIGACIÓN
Drift de esquema en producción (Firebird)	Medio	Diff de esquema previo al corte (herramienta lista)
Paridad legal fina de Sueldos no entra en 4 meses	Alto	Alcance acotado; paridad fina como ajuste posterior
Cobertura completa del SPA no entra en 4 meses	Medio	Pantallas core en el período; cobertura total como ajuste posterior
Acoplamiento del 4º sistema (modelos de datos distintos)	Medio	Maestro único de clientes + sync por API; migración nativa posterior
Cambios normativos durante el proyecto	Medio	Conceptos/tablas parametrizables; mantenimiento posterior
Inconsistencia de datos migrados	Alto	Validación con reconciliación financiera, no solo conteo

15. Supuestos y dependencias

- Acceso remoto a la PC de producción (RDP/AnyDesk) y clave **SYSDBA** de Firebird.
- Acceso a la base MySQL del sistema de gestión existente y decisión **acoplar vs. migrar**.
- **Frontend:** SPA nueva que consume la API (stack propuesto Vue 3 + Vite + Tailwind + Pinia; confirmar preferencia Vue/React).
- Alcance de migración: qué empresas y qué rango de períodos.
- Credenciales para integraciones AFIP/ARCA y, si aplica, WhatsApp Business API.
- Confirmación del listado de reportes/formularios oficiales prioritarios.
- Ventana de **freeze** acordada para el corte; servidor en la nube y dominios.

Plan Técnico de Implementación · v2.1 · backend Modux + SPA Vue 3 · documento de referencia para evaluación técnica ·
acompaña a la Propuesta comercial · sujeto a la definición final de alcance.